

INHERITED FROM Helix Constitution

This module is a submodule of a consuming project that includes the Helix Constitution submodule at the parent's `constitution/` path. All rules in `constitution/CLAUDE.md` and the `constitution/Constitution.md` it references (universal anti-bluff covenant Â§11.4, no-guessing mandate Â§11.4.6, credentials-handling mandate Â§11.4.10, host-session safety Â§12, data safety Â§9, mutation-paired gates Â§1.1) apply unconditionally to every change landed here. The module-specific rules below extend them â€” they never weaken any universal clause.

When this file disagrees with the constitution submodule, the constitution wins. Locate the constitution submodule from any arbitrary nested depth using its `find_constitution.sh` helper.

Canonical reference: <https://github.com/HelixDevelopment/HelixConstitution>

CLAUDE.md - The project AI Agent Manual

The project - AI Agent Operating Manual

Version: 1.0.0 **Date:** 2026-04-30 **Scope:** This document guides AI agents working on the project's codebase **Authority:** Cascaded from the parent project root `CLAUDE.md` with project-specific addenda

1. Agent Identity & Purpose

You are an AI agent working on **the project**, an enterprise-grade distributed AI development platform. Your work directly impacts the quality and usability of a production system.

Your mandate: Write real, working, tested code. No simulations. No placeholders. No â€œfor nowâ€” implementations. Every feature you implement **MUST** actually work when a user invokes it.

1.1 Peer Governance Documents (keep in sync)

This `CLAUDE.md` sits alongside several other agent/governance manuals at the repo root. They overlap and must remain consistent: - `CONSTITUTION.md` â€” source of truth for all mandates (CONST-033, CONST-035, CONST-036â€”040, Article XI Â§11.9). When this file conflicts with the Constitution, the Constitution wins. - `AGENTS.md` â€” generic agent manual (40 KB; mirror anti-bluff rules here). - `CRUSH.md`, `QWEN.md` â€” sibling agent manuals for other CLI tools. Cascade rule changes to all of them. - `helix_code/CLAUDE.md`, `helix_qa/CLAUDE.md`, `challenges/CLAUDE.md` â€” submodule-scoped manuals; this root file inherits from them and they inherit from this one.

Article XII Â§12.1 (CONST-042) â€” No-Secret-Leak

No API key, token, password, certificate, or other credential may be committed to any repository owned by HelixDevelopment or vasic-digital. All secrets live in `.env` files (mode 0600) listed in `.gitignore`. Any leak is a release blocker until rotated and post-mortemed.

Article XII Â§12.2 (CONST-043) â€” No-Force-Push

No force push, force-with-lease push, history rewrite, branch deletion of `main/master`, or upstream-overwriting operation may be performed without explicit, in-conversation user approval per operation. Authorization for one push does not extend further. Bypassing hooks / signing / protected-branch rules also requires explicit approval.

3. Project-Specific Architecture

3.1 Technology Stack

- **Language:** Go “root meta-repo on go 1.25.2, inner Go application (helix_code/) on go 1.26. Keep both modules current; do not downgrade.
- **Module IDs:** root dev.helix.code (thin), inner dev.helix.code (full app + transitive deps).
- **HTTP / API:** Gin v1.11.0, gorilla/websocket v1.5.3, gRPC v1.80.0.
- **Persistence:** PostgreSQL 15+ via pgx/v5 + lib/pq; Redis 7+ via go-redis/v9.
- **AuthN/Z:** golang-jwt/v4 v4.5.2, bcrypt/argon2 (golang.org/x/crypto), oauth2.
- **Config / CLI:** Viper v1.21.0, Cobra v1.8.0, pflag v1.0.10, fsnotify v1.9.0.
- **LLM / Cloud:** AWS Bedrock runtime (aws-sdk-go-v2), Azure azcore/azidentity, getzep/zep-go/v3, smacker/go-tree-sitter.
- **UI:** Fyne v2.7.0 (desktop GUI), tviv / tcell/v2 (terminal UI), chromedp (headless browser).
- **Testing:** stretchr/testify v1.11.1.

3.2 Repository Layout – Meta-Repo + Submodules

This repo is a governance/meta-repo, not the Go application. The actual Go binary lives in the `helix_code/` subdirectory (a submodule). When an agent says “`edit internal/auth`,” they almost always mean `helix_code/internal/auth`, not the root `internal/`.

```
helix_code/ # â†“ repo root (governance + s
â”€â”€ CLAUDE.md / AGENTS.md / CONSTITUTION.md / CRUSH.md / QWEN.md #
â”€â”€ Makefile # governance gates only (S
â”€â”€ go.mod # thin root module (dev.he
â”€â”€ helix # Docker facade script (ru
â”€â”€ setup.sh # one-shot: submodule init
â”€â”€ .gitmodules # source of truth for subm
â”€â”€ docker-compose.helix.yml # standalone deployment
â”€â”€ internal/{fix,security,testing,theme} # root-level helpers ONLY
â”€â”€ cmd/security-test/ # root-level security-test
â”€â”€ scripts/ # init-submodules, propaga
â”€, # verify-governance-cascade,
â”€, # demo-all, run-all-tests, â
```

```

â€œâ€œâ€œ docs/ # ARCHITECTURE.md, COMPLETE
â€œ, # bluff-proofing/, llms_veri
â€œ,
â€œâ€œâ€œ helix_code/ â†Œ TRACKED SUBDIRECTORY (NOT a submodule â€œ m
â€œâ€œâ€œ helix_qa/ â†Œ SUBMODULE: QA / challenge-orchestration pla
â€œâ€œâ€œ challenges/ â†Œ SUBMODULE: cross-cutting Challenge bank (Par
â€œâ€œâ€œ containers/ â†Œ SUBMODULE: Docker/container artefacts
â€œâ€œâ€œ Dependencies/ â†Œ SUBMODULES: LLama_CPP, Ollama, HuggingFace_H
â€œâ€œâ€œ security/ â†Œ SUBMODULE: security tooling
â€œâ€œâ€œ Assets/ â†Œ SUBMODULE: logos, themes, brand
â€œâ€œâ€œ github_pages_website/ â†Œ SUBMODULE: marketing site
â€œâ€œâ€œ Example_Projects/ â†Œ reference projects (Aider, Cline, Plan

```

3.2.1 Inner Go application â€œ helix_code/ submodule

```

helix_code/helix_code/ # module dev.helix.code, go 1.
â€œâ€œâ€œ Makefile # real build/test targets
â€œâ€œâ€œ cmd/
â€œ, â€œâ€œâ€œ server/ # HTTP server entry â†Œ'
â€œ, â€œâ€œâ€œ cli/ # CLI client entry â†Œ' b
â€œ, â€œâ€œâ€œ helix-config/ # config tool
â€œ, â€œâ€œâ€œ config-test/ # config validator
â€œ, â€œâ€œâ€œ security-test/, security-fix*/ # security tools
â€œ, â€œâ€œâ€œ performance-optimization*/ # perf tools
â€œâ€œâ€œ internal/ # ~45 packages â€œ the rea
â€œ, â€œâ€œâ€œ auth/ agent/ cognee/ commands/ config/
â€œ, â€œâ€œâ€œ context/ database/ deployment/ discovery/ editor/
â€œ, â€œâ€œâ€œ event/ focus/ hardware/ helixqa/ hooks/
â€œ, â€œâ€œâ€œ llm/ logging/ logo/ mcp/ memory/
â€œ, â€œâ€œâ€œ monitoring/ notification/ performance/ persistence/ proje
â€œ, â€œâ€œâ€œ provider/ providers/ redis/ repomap/ rules/
â€œ, â€œâ€œâ€œ security/ server/ session/ task/ template
â€œ, â€œâ€œâ€œ tools/ verifier/ version/ worker/ workflow
â€œ, â€œâ€œâ€œ adapters/ fix/ testutil/ mocks/ # mocks/
â€œâ€œâ€œ applications/
â€œ, â€œâ€œâ€œ desktop/ (Fyne GUI)
â€œ, â€œâ€œâ€œ terminal-ui/ (tview TUI)
â€œ, â€œâ€œâ€œ ios/ android/ aurora-os/ harmony-os/
â€œâ€œâ€œ tests/
â€œ, â€œâ€œâ€œ e2e/challenges/ # E2E challenge runner (cmd/runner/main.
â€œ, â€œâ€œâ€œ integration/ # gated by `tags=integration`
â€œ, â€œâ€œâ€œ unit/ # mocks ALLOWED here only
â€œ, â€œâ€œâ€œ security/ # security suite
â€œ, â€œâ€œâ€œ performance/ # benchmarks
â€œâ€œâ€œ config/ # YAML configs (dev/, prod/, test/)
â€œâ€œâ€œ docker/ scripts/ shared/ qa-integration/
â€œâ€œâ€œ docker-compose.full-test.yml + .env.full-test # zero-skip int

```

Cardinal rule: if a path in instructions doesnâ€™t start with `helix_code/`, `helix_qa/`, etc., assume it is relative to the inner Go module and prefix with `helix_code/`.

3.3 Historical Bluffs â€” Resolved, Guard Against Regression

The three patterns below were live bluffs in earlier revisions of `helix_code/cmd/cli/main.go`. They have been fixed (verify with `grep -rn "simulate\|For now\|TODO implement\|placeholder" helix_code/cmd/cli/main.go` â€” must return empty). Treat these as canonical anti-pattern examples; if a future change reintroduces any of them, the change is broken regardless of whether tests pass.

BLUFF-001: LLM Generation is Simulated

Location: `helix_code/cmd/cli/main.go` â†’ function `handleGenerate` **Status:** RESOLVED â€” now calls `provider.Generate / GenerateStream` directly. Do not regress. **Code Pattern:**

```
// ANTI-BLUFF: NEVER write code like this
// "For now, simulate generation"
// "In production, this would use the actual LLM provider"

// WRONG - SIMULATION:
response := fmt.Sprintf("Generated response for: %s\n\nThis is a simulated

// CORRECT - REAL IMPLEMENTATION:
resp, err := c.llmProvider.Generate(ctx, req)
if err != nil {
    return fmt.Errorf("generation failed: %w", err)
}
fmt.Println(resp.Text)
```

Agent Rule: When implementing LLM-related code, you MUST make real HTTP calls to real providers. NEVER simulate responses.

3.4 Build & Test Commands

Two Makefiles. The **root** Makefile only runs governance gates; the **inner** `helix_code/` Makefile does real builds and tests. Always know which directory you are in.

Root governance gates (run from repo root):

```
make no-silent-skips          # fail on bare t.Skip() without SKIP-OK marke
make demo-all                 # run every submodule's demo (proves they act
make demo-one MOD=<name>      # run one submodule's demo
make ci-validate-all         # all governance gates in warn-mode
./setup.sh                    # first-time: submodules + system deps + buil
./scripts/init-submodules.sh  # init all submodules
./scripts/propagate-governance.sh  # cascade Constitution/CLAUDE
./scripts/verify-governance-cascade.sh  # confirm anchors present in
./helix start | stop | logs | shell  # Docker facade for the platf
```

Inner application (run from `helix_code/`):

```
make build                    # â†’ bin/helixcode (server)
make verify-compile           # quick compile-only sanity check
make test                     # all unit tests
make test-coverage            # coverage with -race
```

```

make fmt                # gofmt
make lint               # golangci-lint run
make dev                # build + run with config/dev/config.yaml
make prod               # cross-compile linux/macos/windows

```

Full integration / E2E (real PostgreSQL + Redis + Ollama via docker-compose):

```

make test-infra-up      # start docker-compose.full-t
make test-infra-status  # check stack health
make test-full          # ALL tests, ZERO skips
make test-unit-full / test-integration-full / test-e2e-full / test-security
make test-verifier-unit / test-verifier-integration / test-verifier-challenges
make test-infra-down    # tear down stack + volumes

```

Containerized builds (no host Go required):

```

make container-builder-image # build the builder image once
make container-build         # build inside container
make container-test          # test inside container
make container-shell         # interactive shell in builder
make container-release       # full release in container

```

Single-test invocation (inner module):

```

cd <project_root>
go test -v -run TestJWTGenerate ./internal/auth #
go test -v -tags=integration -run TestAPI_CreateTask ./tests/integration/... #
go test -v -count=1 ./internal/verifier/... #
go test -v -race -coverprofile=cover.out ./internal/llm #

```

E2E challenges (real, end-to-end, runtime evidence required):

```

cd helix_code/tests/e2e/challenges && go run cmd/runner/main.go -all
# Or root-level cross-cutting Challenges:
cd Challenges && make <target>

```

Anti-bluff smoke check (must always pass):

```

grep -rn "simulated\\|for now\\|TODO implement\\|placeholder" \
  helix_code/internal helix_code/cmd && echo "BLUFF FOUND" || echo "clean"

```

Platform / mobile builds (inner module):

```

make desktop / desktop-nogui / desktop-linux / desktop-macos / desktop-windows
make mobile-init && make mobile-ios && make mobile-android
make aurora-os && make harmony-os

```

BLUFF-002: Model Listing is Hardcoded

Location: `helix_code/cmd/cli/main.go` â†’ function `handleListModels` **Status:** RESOLVED â€” must continue to query `c.providerManager.GetProviders()` per CONST-036/037 (LLMsVerifier is the single source of truth). **Correct Pattern:**

```

func (c *CLI) handleListModels(ctx context.Context) error {
    // Query ALL configured providers

```

```

    for name, provider := range c.providerManager.GetProviders() {
        models, err := provider.GetModels()
        if err != nil {
            log.Printf("Warning: failed to list models from %s: %v", name,
                err)
            continue
        }
        // Display real models
        for _, model := range models {
            fmt.Printf("%s/%s: %s (context: %d)\n", name, model.ID, model.
                Name)
        }
    }
    return nil
}

```

BLUFF-003: Command Execution is Simulated

Location: helix_code/cmd/cli/main.go â†’ function handleCommand **Status:** RESOLVED â€” must continue to use os/exec via exec.CommandContext and surface real exit codes. Never replace with print-and-sleep. **Correct Pattern:**

```

func (c *CLI) handleCommand(ctx context.Context, command string) error {
    // ANTI-BLUFF: Actually execute the command
    cmd := exec.CommandContext(ctx, "sh", "-c", command)
    cmd.Dir = c.workingDirectory

    output, err := cmd.CombinedOutput()

    fmt.Printf("Exit code: %d\n", cmd.ProcessState.ExitCode())
    fmt.Printf("Output:\n%s\n", string(output))

    return err
}

```

4. Code Patterns for Agents

4.1 Interface-Driven Design

```

// Define the contract
type Provider interface {
    Generate(ctx context.Context, req *GenerateRequest) (*GenerateResponse,
        error)
    GetModels() ([]Model, error)
    HealthCheck(ctx context.Context) error
}

// Implement with REAL behavior
type OllamaProvider struct { ... }
func (p *OllamaProvider) Generate(ctx context.Context, req *GenerateRequest) (*GenerateResponse, error) {
    // Make REAL HTTP call
    // NO simulation
}

```

4.2 Manager Pattern

```
type TaskManager struct {
    db      TaskRepository
    mu      sync.RWMutex
    tasks   map[uuid.UUID]*Task
}

func (m *TaskManager) Create(ctx context.Context, task *Task) error {
    m.mu.Lock()
    defer m.mu.Unlock()

    // Persist to REAL database
    if err := m.db.Save(ctx, task); err != nil {
        return fmt.Errorf("failed to save task: %w", err)
    }

    m.tasks[task.ID] = task
    return nil
}
```

4.3 Error Handling

```
// Package-level errors
var (
    ErrInvalidCredentials = errors.New("invalid credentials")
    ErrTokenExpired       = errors.New("token expired")
)

// Contextual wrapping
func (s *Service) DoSomething(ctx context.Context) error {
    result, err := s.db.Query(ctx)
    if err != nil {
        return fmt.Errorf("failed to query database for user %s: %w", user, err)
    }

    if err := s.process(result); err != nil {
        return fmt.Errorf("failed to process query result: %w", err)
    }

    return nil
}
```

4.4 Testing Pattern (Unit)

```
func TestService_DoSomething(t *testing.T) {
    tests := []struct {
        name      string
        setup     func(*mockRepository)
        wantErr   bool
    }{
        {
            name: "success",
```

```

        setup: func(m *mockRepository) {
            m.On("Query", mock.Anything).Return(&Result{Data: "test"},
        },
        wantErr: false,
    },
    {
        name: "database_error",
        setup: func(m *mockRepository) {
            m.On("Query", mock.Anything).Return(nil, errors.New("conne
        },
        wantErr: true,
    },
}

for _, tt := range tests {
    t.Run(tt.name, func(t *testing.T) {
        repo := new(mockRepository)
        tt.setup(repo)

        svc := NewService(repo)
        err := svc.DoSomething(context.Background())

        if tt.wantErr {
            require.Error(t, err)
        } else {
            require.NoError(t, err)
        }

        repo.AssertExpectations(t)
    })
}
}

```

4.5 Testing Pattern (Integration - NO MOCKS)

```

func TestAPI_CreateTask_Integration(t *testing.T) {
    if testing.Short() {
        t.Skip("Integration test skipped in short mode")
    }

    // Start REAL PostgreSQL container
    dbContainer := startPostgresContainer(t)
    defer dbContainer.Terminate(context.Background())

    // Connect to REAL database
    db := connectToPostgres(dbContainer)

    // Initialize REAL service
    taskMgr := task.NewManager(db)

    // ANTI-BLUFF: Test with REAL data
    task, err := taskMgr.Create(context.Background(), &task.Task{
        Title: "Integration Test Task",
    })
}

```



```

}))

require.NoError(t, err)
require.NotZero(t, task.ID)

// ANTI-BLUFF: Verify it REALLY exists in database
persisted, err := taskMgr.Get(context.Background(), task.ID)
require.NoError(t, err)
require.Equal(t, "Integration Test Task", persisted.Title)
}

```

5. Anti-Bluff Checklist for Every Task

Before marking any task complete, verify:

- ☐ **No simulation:** Code doesn't contain "simulate", "for now", "TODO implement", "placeholder"
 - ☐ **Real HTTP calls:** API clients make actual HTTP requests with real bodies
 - ☐ **Real database operations:** Database code uses real queries, not in-memory maps (unless explicitly caching)
 - ☐ **Real process execution:** Shell/command execution uses `os/exec`, not `fmt.Printf + time.Sleep`
 - ☐ **Real file operations:** File tools use `os.ReadFile/os.WriteFile`, not mock in-memory buffers
 - ☐ **Test validates reality:** Tests check actual behavior, not just function call counts
 - ☐ **Challenge validates end-to-end:** Challenge script exercises the complete user workflow
 - ☐ **Documentation example works:** README example executes successfully when copy-pasted
 - ☐ **No bare skips:** All `t.Skip()` have `SKIP-OK: #<ticket>` markers
 - ☐ **Evidence pasted:** Commit/PR contains actual terminal output from real execution
-

6. Common Anti-Patterns to Avoid

ANTI-PATTERN 1: The Simulation Trap

```

// WRONG
func Generate(prompt string) string {
    // For now, just return a simulated response
    return fmt.Sprintf("Generated: %s", prompt)
}

```

```
// CORRECT
func (p *Provider) Generate(ctx context.Context, req *GenerateRequest) (*G
    resp, err := p.client.Post(p.endpoint, req)
    if err != nil {
        return nil, fmt.Errorf("generation request failed: %w", err)
    }
    return parseResponse(resp)
}
```

ANTI-PATTERN 2: The Hardcoded List

```
// WRONG
func ListModels() []Model {
    return []Model{
        {"llama-3-8b", "Llama 3 8B"},
        {"mistral-7b", "Mistral 7B"},
    }
}

// CORRECT
func (p *Provider) GetModels() ([]Model, error) {
    resp, err := p.client.Get(p.baseURL + "/api/tags")
    if err != nil {
        return nil, err
    }
    return parseModelList(resp)
}
```

ANTI-PATTERN 3: The Stub Interface

```
// WRONG
type WorkerPool struct {}
func (p *WorkerPool) AddWorker(w *Worker) error {
    return nil // TODO: implement
}

// CORRECT
func (p *SSHWorkerPool) AddWorker(ctx context.Context, w *SSHWorker) error {
    client, err := ssh.Dial("tcp", w.Host, w.SSHConfig)
    if err != nil {
        return fmt.Errorf("failed to connect to worker %s: %w", w.Host, err)
    }
    defer client.Close()

    // Verify worker has helix binary
    session, err := client.NewSession()
    if err != nil {
        return fmt.Errorf("failed to create SSH session: %w", err)
    }
    defer session.Close()

    // Actually test the worker
    output, err := session.Output("which helix || echo 'NOT_INSTALLED'")
}
```

```

    if strings.Contains(string(output), "NOT_INSTALLED") {
        // Auto-install
        if err := p.installWorker(ctx, client); err != nil {
            return fmt.Errorf("failed to install worker: %w", err)
        }
    }

    p.workers[w.Hostname] = w
    return nil
}

```

7. Working with Submodules

The project has 80+ submodules. When working with them:

1. **Check governance:** Does the submodule have Constitution.md / CLAUDE.md / AGENTS.md?
 2. **Add if missing:** Create governance files referencing parent
 3. **Verify builds:** Does the submodule actually compile?
 4. **Test integration:** Does the project integration with this submodule work?
-

8. Emergency Procedures

If You Discover a Bluff

1. STOP working on dependent features
2. Document the bluff in docs/issues/BLUFFS.md
3. Write a Challenge that reproduces the bluff
4. Fix the bluff
5. Verify the Challenge now passes
6. Update documentation to reflect reality

If a Test Passes But Feature Doesn't™ Work

1. The test is a bluff - tighten it
 2. Add assertions that verify actual output quality
 3. Add anti-bluff checks (no "œsimulatedœ in responses)
 4. Run the test against real infrastructure
 5. Verify it FAILS with the broken code
 6. Then fix the code
-

9. Reference Commands

The full command catalog lives in [§3.4 Build & Test Commands](#). The block below is only the smoke-test you should run before claiming any change is done.

```

# 1. Compiles?
cd <project_root> && make verify-compile

```

```
# 2. Unit tests (mocks allowed only here)
cd <project_root> && go test -count=1 ./...

# 3. Anti-bluff scan
grep -rn "simulated\\|for now\\|TODO implement\\|placeholder" \
  helix_code/internal helix_code/cmd && echo "BLUFF FOUND" || echo "clean"

# 4. Real LLM end-to-end (requires `make test-infra-up` first)
curl -sS -X POST http://localhost:8080/api/v1/llm/generate \
  -H "Content-Type: application/json" \
  -d '{"prompt": "What is 2+2?", "model": "llama3.2"}'
# Must return real AI output, not "simulated response".

# 5. Governance still cascading?
./scripts/verify-governance-cascade.sh
```

10. LLMsVerifier Constitutional Mandates (CONST-036 through CONST-040)

CONST-036: LLMsVerifier Single Source of Truth

LLMsVerifier is the sole authoritative source for model metadata, provider metadata, verification status, and scoring data. NO hardcoded model lists. NO simulated discovery.

CONST-037: Model Provider Anti-Bluff Guarantee

Every model displayed to users MUST be verified by LLMsVerifier within 24h. Integration tests MUST use real verifier data, not mocks.

CONST-038: Real-Time Model Status Accuracy

Model status MUST reflect verifier state within 60s. Poll interval $\hat{\approx}$ 60s if push unavailable.

CONST-039: All Providers Integration Mandate

The project MUST integrate with all verifier-supported providers: OpenAI, Anthropic, Gemini, DeepSeek, Groq, Mistral, xAI, OpenRouter, Ollama, Llama.cpp.

CONST-040: Capability Integration Mandate

MCP, LSP, ACP, Embedding, RAG, Skills, and Plugins capability flags MUST be sourced from verifier `VerificationResult`. NO hardcoded capability flags.

11. Contact & Escalation

- **Bluff reports:** docs/issues/BLUFFS.md
- **Bug fixes:** docs/issues/fixed/BUGFIXES.md
- **Architecture questions:** docs/ARCHITECTURE.md
- **Emergency:** Create a Challenge that reproduces the issue

-
- **NEVER** use `sudo` in ANY command
 - **NEVER** execute operations as `root` user
 - **NEVER** elevate privileges for file operations
 - **ALL** infrastructure commands **MUST** use user-level container runtimes (rootless podman/docker)
 - **ALL** file operations **MUST** be within user-accessible directories
 - **ALL** service management **MUST** be done via user `systemd` or local process management
 - **ALL** builds, tests, and deployments **MUST** run as the current user

Why This Matters

- **Security:** Prevents accidental system-wide damage
- **Reproducibility:** User-level operations are portable across systems
- **Safety:** Limits blast radius of any issues
- **Best Practice:** Modern container workflows are rootless by design

When You See SUDO

If any script or command suggests using `sudo`: 1. STOP immediately 2. Find a user-level alternative 3. Use rootless container runtimes 4. Modify commands to work within user permissions

VIOLATION OF THIS CONSTRAINT IS STRICTLY PROHIBITED.